Lab 2: Designing and Simulating Combinational Logic Blocks

In this lab, you will design and simulate various combinational logic blocks based on the concepts covered in **Lectures 6 and 7**. By the end of this lab, you should be able to design, implement, and simulate basic combinational circuits such as multipliers, multiplexers, and shifters/rotators.

You can find the code snippets at https://github.com/silagokth/il1332ht24lab2.

For each problem, start by drawing a schematic of your design, then write the SystemVerilog code and a testbench to validate your design. Your testbench should cover all possible cases.

Your designs should be structural, and there's no need to account for delays.

A. Multiplier

In Lecture 6, you learned how to design and model the 4x4 multiplier using Ripple Carry Adders (RCA). For this lab:

a. Generalize the multiplier to the MxN multiplier.

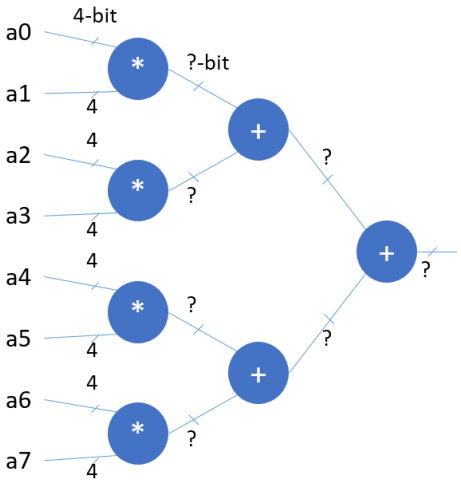
```
module mult_mnbit #(parameter M = 4, parameter N = 4)(
      input logic[M-1:0] a,
      input logic [N-1:0] b,
      output logic [M+N-1:0] product
);
//
endmodule
```

b. Extend it to deal with signed numbers.

c. (**Bonus**) Use the designed multiplier and an RCA to multiply, add, and calculate x as described below.

$$x = a0 \times a1 + a2 \times a3 + a4 \times a5 + a6 \times a7$$

All **'a'** numbers are unsigned 4-bit. Make sure that no information is lost in the final result. You need to define the appropriate number of bits for the result x.




```
module mult_add(
      input logic [3:0] a0,
      input logic [3:0] a1,
      input logic [3:0] a2,
      input logic [3:0] a3,
      input logic [3:0] a4,
      input logic [3:0] a5,
      input logic [3:0] a6,
      input logic [3:0] a7,
      output logic [?:0] x);
      //
endmodule
```

B. Multiplexers

Design and simulate an N-bit RCA using muxes to implement Sum and Carry-Out logic.

Hint: Use the generic mux structures as covered in the class to build sum and carry logic.

```
module rca_nbit #(parameter N = 4)(
      input logic [N-1:0] a,
      input logic [N-1:0] b,
      input logic cin,
      output logic [N-1:0] sum,
      output logic cout
);
```

```
//
endmodule
```

C. Shifter and rotator

Design and simulate a combinational block "**shifter_rotator**" that performs shift and rotate operations. Shifting should be logical, and the operation is controlled by a 2-bit select input as described below:

S_1	S_0	Operation
0	0	Shift left
0	1	Shift right
1	0	Rotate left
1	1	Rotate right

```
module shifter_rotator(
      input logic[3:0] x,
      input logic[1:0] select,
      output logic[3:0] y
      );
//
endmodule
```